EmPath Implementation Roadmap

Purpose: build a Python prototype that compares residential access to temples, groceries and pharmacies using distance-only and environmental-friction routes.

Scope: Plano ZIP areas 75025, 75024, 75023 and 75074, represented by Census ZCTA boundaries (U.S. Census Bureau, n.d.). Your budget: 20 hours this week, including a two-hour team meeting and the nine-hour hackathon.

Minimum demonstration: approximately 5–10 verified residential origins per ZIP, individual destinations in three categories, one environmental scenario and one reused or adapted GeoAI component. Keep the four-area scope; reduce sample size or optional features if needed.

Workflow at a Glance

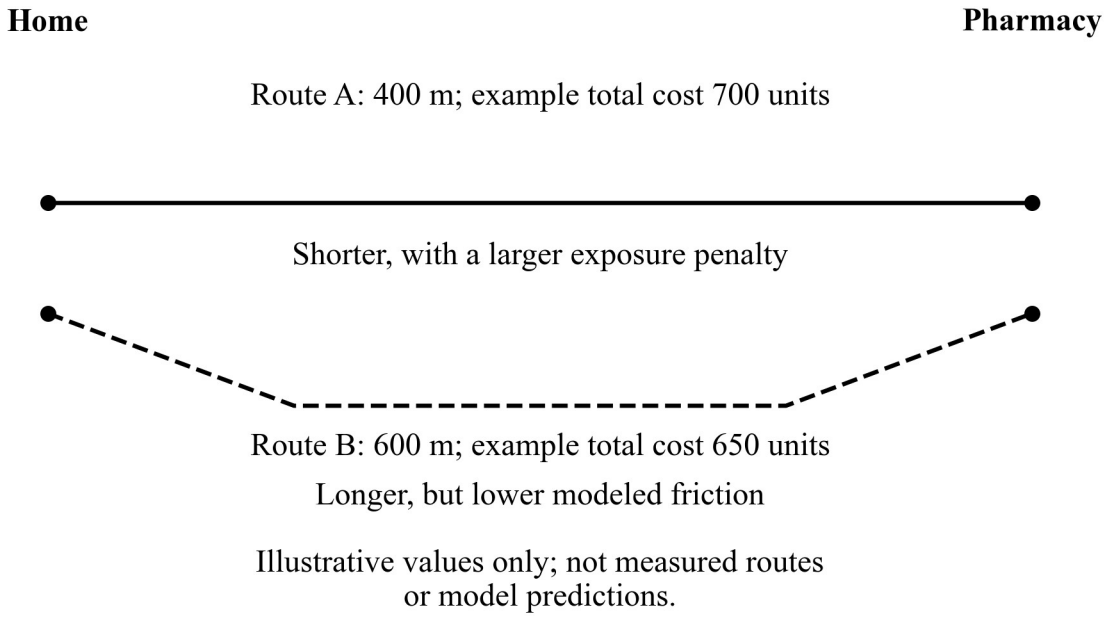
Step	Time	Deliverable
1. Inventory	1 h	Sources and coverage checklist
2. Clean inputs	2 h	Clean data and quality report
3. Build network	2.5 h	Walking graph and connection audit
4. Add features	2 h	Edge features and data dictionary
5. Adapt GeoAI	3 h	Model adapter and model card
6. Route and rank	2 h	Routes and residential rankings
7. Evaluate and map	2.5 h	Comparison results and interactive map
8. Package	3 h	README, slides and backup

How the Algorithms Work

Think of EmPath as a walking map with a burden score attached to each small path segment. The system uses six separate operations:

1. Build the map. A graph is a collection of dots and connecting lines. Dots represent junctions or access points; lines represent walkable segments. Homes and destinations attach to this network.
2. Describe each segment. Record distance and supported environmental information, such as a canopy estimate. A missing connection remains a gap; it is not repaired merely because two dots are close.
3. Use the existing AI model. The model estimates the quantity it was designed to predict, such as an environmental feature. A behavioral model may instead estimate shade–distance preferences. Neither model can infer every walking need from a map alone.
4. Assign walking costs. Convert supported estimates into a nonnegative segment cost, alongside distance. A cost is a comparison score, not automatically minutes or medical risk. Document any manually chosen penalty.
5. Choose a route. Dijkstra's algorithm tracks the lowest accumulated cost found while exploring connected paths. It returns a minimum-cost route; it does not need to list every possible route. The same algorithm can optimize distance or friction simply by changing the costs.
6. Compare homes. For each home, find the lowest-cost reachable temple, grocery and pharmacy. Add those three category costs with equal weights for the prototype. Lower totals rank better; flag missing services rather than quietly ignoring them.

Example: the shorter route below has a larger exposure penalty. The illustrative totals are 400 + 300 = 700 for Route A and 600 + 50 = 650 for Route B. Distance-only routing selects A; friction routing selects B. Real penalties require evidence or an explicit scenario assumption.



1 Inventory Sources and Coverage

- Open EmPath.code-workspace in VS Code. Select .venv/bin/python and the EmPath Jupyter kernel. Read README.md and docs/CONFIRMED_SCOPE.md.
- List boundaries, walking infrastructure, residential origins, destinations, environmental features and model inputs. For each ZIP, mark coverage as downloaded, partial, available online or unverified.
- Record provider, source URL, vintage, resolution, format, reuse terms, intended use and local file. Reuse scripts/audit_walking_coverage.py for walking coverage.

Tools: pandas, GeoPandas, requests. **Save:** sources table and four-area coverage checklist. **Done when** every required input has a status and next action.

2 Download and Clean Essential Inputs

- Run .venv/bin/python scripts/clean_data.py --zipcodes 75025 75024 75023 75074. Preserve raw snapshots; inspect the generated quality report (City of Plano, n.d.-a, n.d.-b, n.d.-c, n.d.-d).
- Check IDs, geometry, duplicates and missing values. Use EPSG:26914 for metric operations and EPSG:4326 for map display.
- Use Plano address points for a small verified residential sample. Parcels support checking but do not prove residential use or pedestrian entrances.
- Query OSM destination candidates: pharmacies, supermarkets/groceries and appropriate places of worship. Verify names, categories and access points. Missing tags do not prove that a facility is absent.

Tools: GeoPandas, Shapely, pandas, requests, OSMnx. **Save:** cleaned GeoPackages, origins.gpkg, destinations.gpkg and quality_report.md. **Done when** all four areas have documented coverage or explicit gaps.

3 Build and Audit the Pedestrian Network

- Use OSMnx graph_from_polygon with network_type="walk" and retain_all=True. Start with the four-ZCTA union plus roughly 500 m surrounding context; expand if routes are truncated (OSMnx contributors, n.d.).
- Save the original GraphML. Compare with Plano sidewalk surfaces and trails. Sidewalk polygons are supporting data, not ready-made routing edges.
- Connect origins and destinations through plausible access points. Audit nearest-node connectors for fences, inaccessible entrances and major crossings. Record components and disconnected origins.
- Inspect at least one route per ZIP. Add municipal paths only after checking access, topology and duplicates; preserve unreachable results.

Tools: OSMnx, NetworkX, GeoPandas, Folium. Reference: Street Shadow for graph-to-route structure (jlgonrod, n.d.); UNA for origin/destination architecture (City Form Lab, n.d.). Save: walking_graph.graphml, access_connectors.gpkg and network_audit.md.

4 Add Environmental Features

- Acquire a manageable clip of the Texas 2021 10 m canopy raster. Sample a documented corridor around each edge; start with a 10 m buffer and record that choice (Diamond et al., 2023).
- Save edge ID, length_m, canopy_fraction, valid_pixel_fraction, source vintage and verified crossing constraints. Check ranges and missingness by ZIP; preserve NoData.
- Treat canopy as an environmental proxy, not time-specific shade or measured heat stress. Keep Tree Equity Score values as neighborhood context. Add weather, traffic, imperviousness or shadows only after core features work (American Forests, n.d.; Wolf et al., 2025).

Tools: Rasterio, NumPy, GeoPandas, Shapely. Reference: CoolWalks for shade-sensitive costs (Wolf et al., 2025). Save: edge_features.gpkg and DATA_DICTIONARY.md. Done when every scoring feature has units, provenance and coverage checks. See Rasterio contributors (n.d.) for raster-processing documentation.

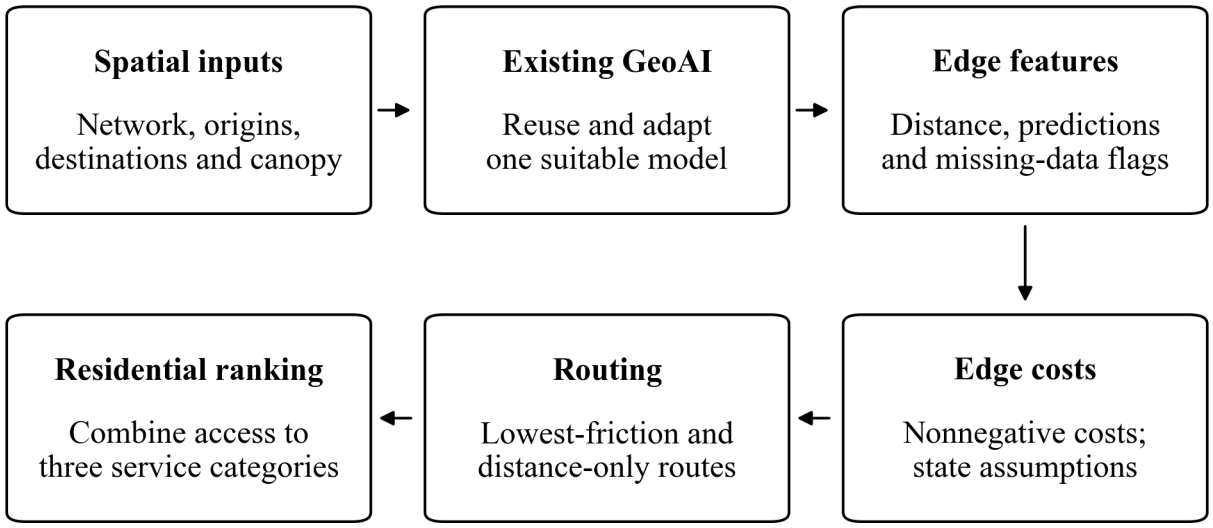
5 Reuse or Adapt One GeoAI Component

Choose one feasible approach: integrate an existing environmental predictor with compatible public inputs, or adapt a released shade–distance route-choice workflow. New model architecture or training from scratch is not required.

- Check that the chosen method meets the hackathon's GeoAI requirement. Record model/version, license, input features, predicted quantity, original training geography and local adaptation.
- For the Singapore study, inspect the public data and reproduce its released analysis before modifying it. Keep participants grouped when fitting and evaluating. Behavioral calibration is not automatically deep learning (Melnikov et al., 2022).
- Use XGBoost or LightGBM only if a suitable existing workflow and meaningful observed labels are available. Install only the selected model's dependencies.
- Keep route-level predictions separate from additive edge costs. Identify manually specified penalties as assumptions; do not claim learned preference weights for unobserved factors.

Save: model adapter or notebook and MODEL_CARD.md. **Done when** the model runs, outputs are checked and reuse is explicit. A precomputed ML canopy product is an input, not proof of a newly implemented model; record any unresolved GeoAI gap.

How the Components Connect



6 Calculate Routes and Residential Rankings

- Define nonnegative additive edge costs with explicit units. Separate model-derived terms from scenario assumptions. Applying sunlight preferences to canopy requires a disclosed transfer assumption.
- Use NetworkX Dijkstra-based routing for distance-only and environmental-friction scenarios. Find the lowest-cost reachable destination in each of the three categories for each origin (NetworkX developers, n.d.).
- Rank origins by an equal-weight sum of category costs. Flag missing or unreachable categories; do not present incomplete origins as fully accessible. Destinations need not occur inside every ZIP.

Tools: NetworkX, OSMnx, pandas, GeoPandas. References: Street Shadow, CoolWalks and Walk Score's amenity framing (Walk Score, n.d.). Save: routes.gpkg and residential_rankings.csv with origin/ZIP, destination IDs, distances, category costs, total cost and reachability.

7 Evaluate and Build the Demonstration

- Audit at least one route per ZIP and one major crossing. Inspect connectivity, access and suspicious jumps.
- Compare a baseline/alternative route pair using distance, canopy proxy and modeled cost. Test two plausible penalty settings and record ranking changes. Report the result even if routes do not improve.
- Evaluate the reused model separately. Performance on observations outside Plano does not establish accuracy for Plano pedestrians.
- Build one Folium map with boundaries, origins, destinations, ranked-origin popups and route alternatives. Precompute results; add Streamlit only if essential and time remains (Folium contributors, n.d.).

Tools: Folium, pandas, matplotlib and model-specific evaluation tools. References: LA sidewalk heat study for research design (Colaninno et al., 2025); City Health Dashboard for communication (City Health Dashboard, n.d.). Save: empath_demo.html, evaluation.csv and LIMITATIONS.md. Done when a viewer can explain a ranking; prepare screenshots because basemap tiles may need internet.

8 Package and Present

- Update README.md with installation, interpreter/kernel, execution order, download instructions, attribution and tested model/library versions. Verify the demo from a fresh session.
- Prepare five-minute slides: problem and user; data and scope; graph and GeoAI; demonstration and finding; limitations and next steps. Save a backup recording or screenshots.
- Hand off sources/coverage, clean-data provenance, graph/features, model adapter/card, routes/rankings, evaluation, demo, code and slides. Reuse generated outputs rather than write a separate long report.

Completion check: the demo loads reproducibly, recommendations are explainable, gaps are disclosed and the presentation fits five minutes. Environmental accessibility does not certify safety or medical suitability.

Working Rules

- Your 20-hour budget includes nine hours of individual preparation, a two-hour team meeting and nine hours at the hackathon. Use the personal schedule below to allocate your time.
- Python is the reproducible pipeline: VS Code, Jupyter and a browser. ArcGIS Pro is optional for a teammate’s review; exchange GeoPackage/GeoJSON files.
- Parmukh supports GIS quality checks and model evaluation, mainly on hackathon day. You own the Python pipeline, integration, map and presentation. See the team plan below.
- Future expansion: local preference validation, detailed crossing/traffic data, time-dependent heat/shade and point-to-point lowest-friction navigation.

Your 20 Hour Work Roadmap

Total: 9 hours of individual preparation + 2 hours meeting + 9 hours hackathon = 20 hours. The step estimates on page 1 total 18 hours of individual work; the meeting adds two hours. Parmukh's 12 hours are a separate contribution, with the same meeting counted once for each person.

Before the Hackathon 9 hours

1. Sources and setup — 1 hour. Open the workspace and confirm the Python environment. Complete the sources and four-ZIP coverage checklist. Deliverable: a clear list of available inputs and gaps (Step 1).

2. Clean essential data — 2 hours. Prepare boundaries, walking data, a small residential sample and verified destinations. Check IDs, missing values and coordinate systems. Deliverable: cleaned layers and quality report (Step 2).

3. Build the walking graph — 2.5 hours. Build and save the network; attach origins and destinations; flag disconnected locations and questionable connectors. Deliverable: GraphML, access connectors and draft network audit (Step 3).

4. Add environmental features — 2 hours. Sample canopy, preserve missingness and document units and provenance. Deliverable: edge features and data dictionary (Step 4).

5. Prepare the GeoAI adapter — 1.5 hours. Select a feasible existing model or released workflow, check its inputs and license, and attempt the first run. Deliverable: runnable starting notebook or a documented blocker (first half of Step 5).

Team Meeting 2 hours

Meet once the graph and first model attempt are ready. Review scope and user needs, data/network gaps, GeoAI and evaluation, then task ownership: 30 minutes each. Give Parmukh saved layers and a prioritized issue list. Deliverable: TEAM_HANDOFF.md and agreed hackathon tasks. If an output is unfinished, review the blocker instead.

At the Hackathon 9 hours

Hours 0–1.5: finish the GeoAI adapter and model card (complete Step 5). Parmukh begins GIS review using your saved graph and layers.

Hours 1.5–3.5: calculate distance-only and friction routes; rank residential origins (Step 6). Deliverable: routes and rankings. Send a comparison pair to Parmukh for evaluation.

Hours 3.5–6: build the Folium demo, compare scenarios and integrate priority audit findings (Step 7). Deliverable: interactive map, evaluation results and limitations. Parmukh completes GIS review and model/results checks.

Hours 6–9: freeze the demo, finish README and five-minute slides, save a backup and rehearse (Step 8). Parmukh uses the final two hours for integration and rehearsal support. Deliverable: reproducible demo and presentation package.

Time Limit Rule

Protect the final three hours for packaging and presentation. Reduce sampled origins or optional layers if needed; keep all four ZIP areas represented. The minimum demo needs a runnable, documented GeoAI component, explainable routes and disclosed data gaps.

Team Roles and Handoffs

Keep the same prototype and four-ZIP scope. Parmukh contributes approximately 12 hours total, including a two-hour team meeting. The allocation below is a proposed schedule; only one hour of individual preparation is expected before hackathon day.

1 Shared Planning Meeting 2 hours each

Review the current demo, four-area coverage and model approach. Spend 30 minutes on scope and user needs, 30 on graph and data gaps, 30 on GeoAI and evaluation, and 30 on owners and handoffs. Save decisions in docs/TEAM_HANDOFF.md. Count these two hours inside both people's budgets.

2 Parmukh Preparation 1 hour

Read the roadmap, coverage checklist and model card. Identify the three highest-priority GIS or modeling risks and add comments to TEAM_HANDOFF.md. No new dataset or model is required before the event.

3 Parmukh GIS Review 4 hours on hackathon day

Review outputs from Steps 2–4: coordinate systems, missing features by ZIP, disconnected origins, destination entrances and major crossings. Check at least one route per ZIP. Return network_audit.md and a small GeoPackage or CSV of flagged locations, suggested fixes and unresolved issues.

4 Parmukh Model and Results Review 3 hours on hackathon day

Review Steps 5–7: model inputs, output meaning, nonnegative edge costs and separation of learned terms from assumptions. Check one distance-versus-friction route pair and two penalty settings. Return evaluation.csv and concise additions to MODEL_CARD.md and LIMITATIONS.md. If usable labels exist, report the appropriate validation metric; otherwise report sensitivity and data checks.

5 Parmukh Integration and Rehearsal 2 hours on hackathon day

Help resolve priority defects, verify the final map and review the five-minute story. Return a short list of checked results and remaining limitations. Total: 2 hours meeting + 1 preparation + 4 GIS review + 3 evaluation + 2 integration = 12 hours.

Your Implementation Priorities

Own the runnable network, GeoAI adapter, routing, map and slides. Follow the 20-hour personal schedule below and share outputs with Parmukh as they become available. His review runs alongside your implementation; it does not add hours to your budget.

Handoff Rules

Use a stable snapshot of GraphML, GeoPackage and CSV outputs, with units and file paths. Parmukh can review in Python or ArcGIS Pro; return portable files and documented findings. You integrate changes into the Python pipeline. If he is unavailable before the event, continue the core build and queue his review for hackathon day.

References

American Forests. (n.d.). *Tree Equity Score methods and data*. https://www.treeequityscore.org/methodology?tab=data-download

City Form Lab. (n.d.). *UNA urban network analysis* [Computer software]. GitHub. https://github.com/City-Form-Lab/urban_network_analysis

City Health Dashboard. (n.d.). *Walkability*. https://www.cityhealthdashboard.com/metric/walkability

City of Plano. (n.d.-a). *Address points* [Data set]. https://maps.planogis.org/arcgiswad/rest/services/OpenData/AddressPoints/FeatureServer/0

City of Plano. (n.d.-b). *Parcels* [Data set]. https://maps.planogis.org/arcgiswad/rest/services/OpenData/Parcels/FeatureServer/1

City of Plano. (n.d.-c). *Sidewalks* [Data set]. https://maps.planogis.org/arcgiswad/rest/services/OpenData/Sidewalks/FeatureServer/0

City of Plano. (n.d.-d). *Trail centerlines* [Data set]. https://maps.planogis.org/arcgiswad/rest/services/OpenData/TrailCenterlines/FeatureServer/0

Colaninno, N., Basu, R., Hosseini, M., Alhassan, A., Liu, L., & Sevtsuk, A. (2025). A sidewalk-level urban heat risk assessment framework using pedestrian mobility and urban microclimate modeling. *Environment and Planning B: Urban Analytics and City Science, 52*(5). https://doi.org/10.1177/23998083241280746

Diamond, D., Sunde, M., & Elliott, L. (2023). *Texas statewide landcover, ecological systems, and percentage canopy classifications (10-meter resolution) (2021)* [Data set]. Zenodo. https://zenodo.org/records/8200761

Folium contributors. (n.d.). *User guide*. https://python-visualization.github.io/folium/latest/user_guide.html

jlgonrod. (n.d.). *Street Shadow* [Computer software]. GitHub. https://github.com/jlgonrod/street_shadow

Melnikov, V. R., Christopoulos, G. I., Krzhizhanovskaya, V. V., Lees, M. H., & Sloot, P. M. A. (2022). Behavioural thermal regulation explains pedestrian path choices in hot urban environments. *Scientific Reports, 12,* Article 2441. https://doi.org/10.1038/s41598-022-06383-5

NetworkX developers. (n.d.). *Shortest paths*. https://networkx.org/documentation/stable/reference/algorithms/shortest_paths.html

OSMnx contributors. (n.d.). *User reference*. https://osmnx.readthedocs.io/en/stable/user-reference.html

Rasterio contributors. (n.d.). *Windowed reading and writing*. https://rasterio.readthedocs.io/en/stable/topics/windowed-rw.html

U.S. Census Bureau. (n.d.). *ZIP Code Tabulation Areas* [Data set]. TIGERweb. https://tigerweb.geo.census.gov/arcgis/rest/services/TIGERweb/tigerWMS_ACS2024/MapServer/2

Walk Score. (n.d.). *Walk Score methodology*. https://www.walkscore.com/methodology.shtml

Wolf, H., Vierø, A. R., & Szell, M. (2025). CoolWalks for active mobility in urban street networks. *Scientific Reports, 15*, Article 14911. https://doi.org/10.1038/s41598-025-97200-2